

군집화

정답이 없는 데이터를 끼리끼리 묶는 일.
여기서부터 비지도학습이 시작됩니다.

Q1 지도 vs 비지도

지금까지는 늘 정답(y)이 있었습니다. 스팸인지 아닌지, 신용등급이 몇인지, 판매액이 얼마인지. 그걸 보고 배우는 걸 지도학습이라고 해요.

지도학습

키	몸무게	정답
172	68	A등급
158	50	B등급
180	85	A등급

답지를 보며 규칙을 배운다
분류 · 회귀

비지도학습

키	몸무게	???
172	68	???
158	50	???
180	85	???

답지 없이 스스로 무리를 찾는다
군집화 · 차원축소 · 연관규칙

군집화 = "이 사람들, 몇 가지 유형으로 나눌지?"

유형이 뭔지는 아무도 안 알려줍니다. 데이터가 알려주죠.

Q2 어디에 쓰나

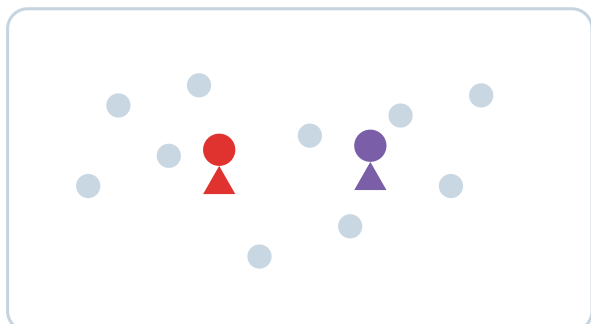
상 황	찾 아 내 는 것
쇼핑몰 고객 1만 명	"알뜰형 / 큰손형 / 뜨내기형" 같은 고객 유형
학급 학생 성적표	비슷한 학습 패턴을 가진 그룹
뉴스 기사 수천 개	비슷한 주제끼리 자동 분류
지역별 데이터	상권이 비슷한 동네끼리 묶기

미리 "고객은 3가지 유형이야"라고 정해준 사람이 없다는 게 핵심입니다. 컴퓨터가 데이터만 보고 스스로 나눕니다.

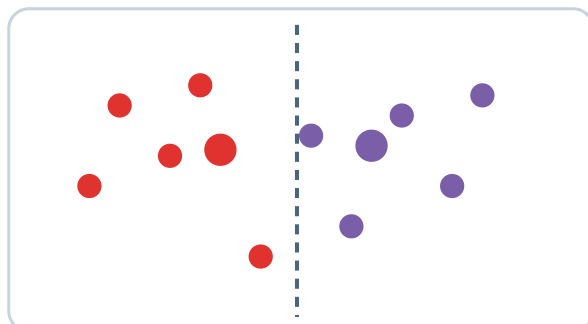
Q3 K-means — 대장 뽑고 편 가르기

가장 많이 쓰는 방법입니다. 이름의 K는 "몇 개로 나눌래?" 라는 뜻이에요. 방법은 놀라울 만큼 단순합니다.

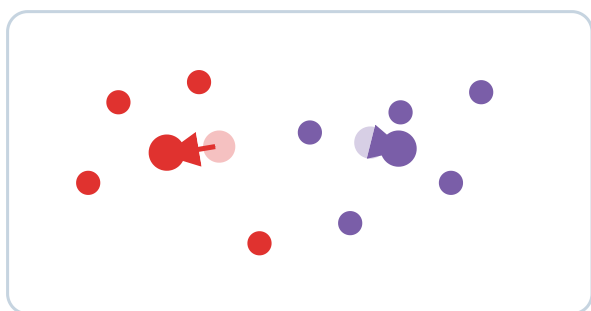
① 대장을 아무 데나 K명 세운다



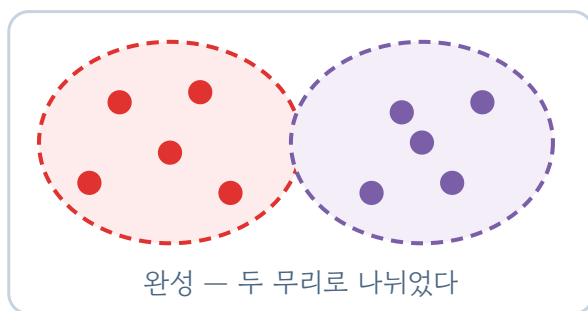
② 각자 가까운 대장 편으로



③ 대장이 자기 편 한가운데로 이동



④ 안 움직일 때까지 ②③ 반복



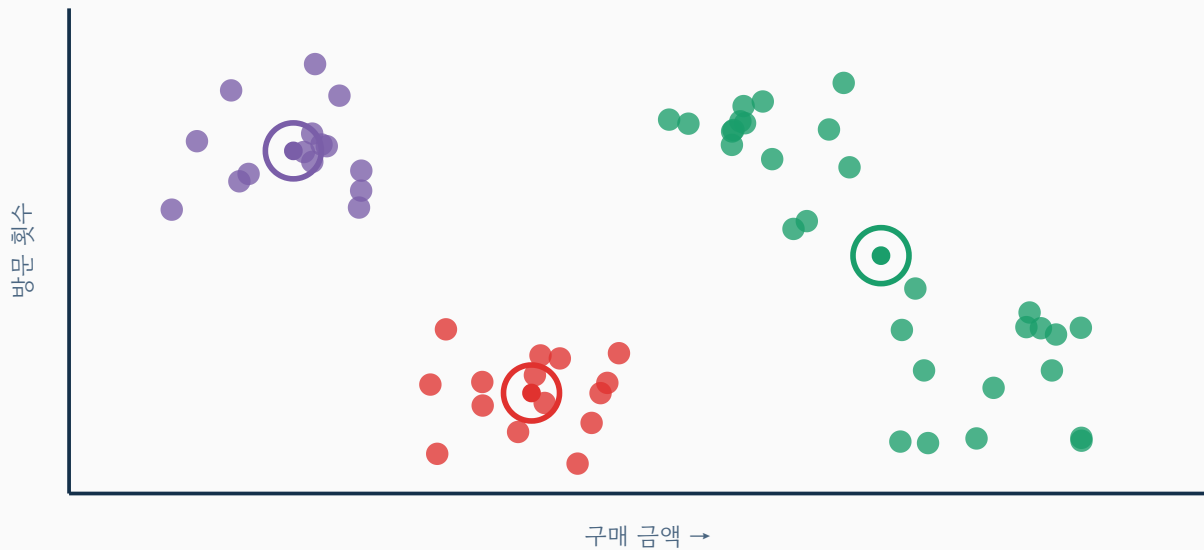
완성 — 두 무리로 나뉘었다

이 단순한 반복만으로 무리가 저절로 만들어집니다.

Q4 직접 나눠보기

K-means 실험실

고객 60명을 **구매 금액**과 **방문 횟수**로 흠어냈습니다. 몇 무리로 나눌지 정해보세요.
큰 동그라미가 각 무리의 대장(중심)입니다.



무리 개수 K

3개

웅침 정도 (작을수록 촘촘)

331

대장 다시 뽑기

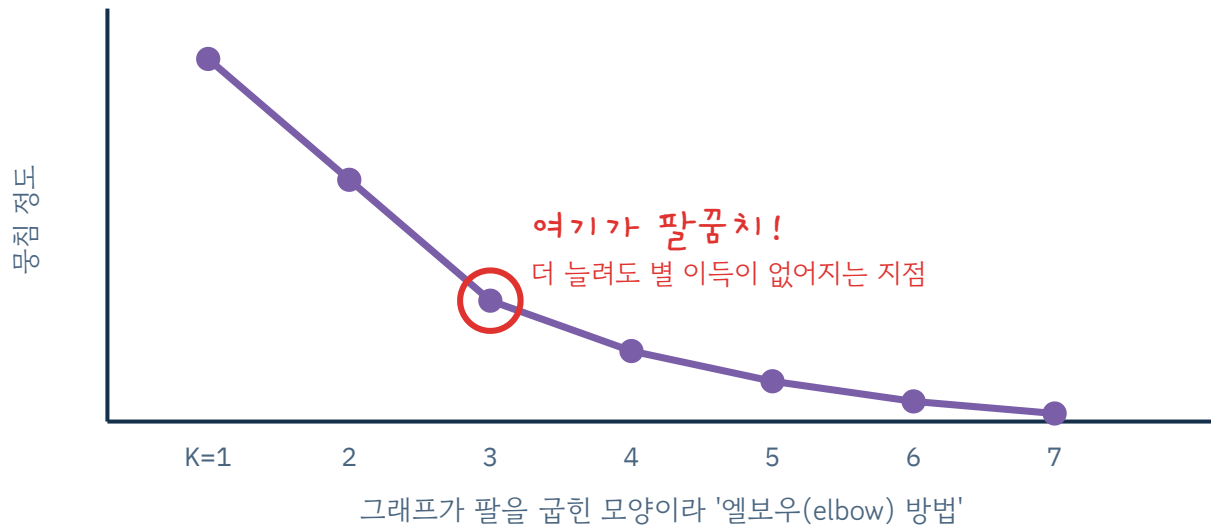
무리들이 잘 떨어져 있습니다. 데이터가 원래 4덩이라서 K=4까지는 자연스러워요.

"대장 다시 뽑기"를 여러 번 눌러보세요. 처음 대장을 어디에 세우냐에 따라 결과가 살짝 달라집니다. 그래서 사이킷런은 여러 번 돌려보고 제일 좋은 걸 자동으로 고릅니다.

Q5 K는 몇으로 해야 하나 — 팔꿈치 찾기

K-means의 가장 큰 고민입니다. 정답이 없으니 "몇 개가 맞다"고 말해줄 사람도 없어요.

K를 늘리면 뭉침 정도는 무조건 좋아집니다. 극단적으로 K를 데이터 개수만큼 늘리면 각자가 자기 무리가 되니까 완벽하게 뭉치죠. 하지만 그건 나눈 게 아니잖아요.



급하게 떨어지다가 갑자기 완만해지는 지점의 K를 고릅니다.

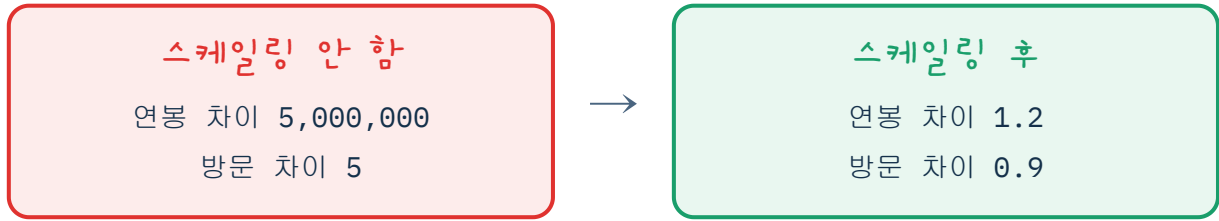
엘보우가 애매하게 나오는 경우도 많습니다. 그럴 땐 **실루엣 점수**라는 다른 지표를 같이 보거나, 아예 "우리 회사는 고객을 4유형으로 관리한다" 같은 현실적인 사정으로 정하기도 합니다. 정답이 없는 문제라서 그렇습니다.

실루엣 점수가 뭔데?

각 점에게 이렇게 물어봅니다 — "너, 네 무리 친구들과는 딱 붙어 있고 옆 무리와는 멀리 떨어졌니?" 그 정도를 $-1 \sim 1$ 점수로 매긴 게 실루엣 점수예요. **1에 가까우면** 무리가 뚜렷하게 잘 나뉜 것, **0 근처면** 무리끼리 애매하게 겹친 것, **음수면** 옆 무리에 잘못 낀 것입니다. 그래서 **K를 2, 3, 4... 로 바꿔가며 돌려보고, 실루엣 점수가 가장 높은 K를** 고르기도 합니다. 엘보우와 함께 보면 더 든든하죠.

Q6 꼭 기억할 것 — 스케일링

K-means는 거리를 재서 편을 가릅니다. 그런데 특성마다 단위가 다르면?



왼쪽에선 방문 횟수를 **아예 없는 것처럼** 취급하게 됩니다

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# 스케일링을 빼먹으면 큰 숫자를 가진 특성이 결과를 지배한다
model = make_pipeline(StandardScaler(), KMeans(n_clusters=3, random_state=0, n_init=10))
labels = model.fit_predict(X)      # 각 손님이 몇 번 무리인지 (0,1,2)

df['무리'] = labels
df.groupby('무리').mean()         # 무리별 평균으로 성격 파악
```

마지막이 진짜 일입니다.

컴퓨터는 "0번 무리, 1번 무리"라고만 알려줍니다. "0번은 알뜰형, 1번은 큰손형"이라고 이름 붙이는 건 사람의 몫이에요. 무리별 평균을 보고 "아, 이쪽은 자주 오는데 조금씩 사는구나" 하고 해석하는 거죠.

Q7 한 장 정리

	분 류	군 집 화
정답(y)	있음	없음
결과	스팸 / 정상처럼 이름이 있는 칸	0번, 1번처럼 이름 없는 무리
채점	정확도, f1	정답이 없어서 채점 불가 — 엘보우·실루엣으로 참고만
사람이 할 일	결과 확인	무리에 이름 붙이고 해석

이것만 기억하면 됩니다

- ① 군집화는 답지 없이 끼리끼리 묶는 일이다
- ② K(무리 개수)는 내가 정해야 하고, 엘보우가 힌트를 준다
- ③ 거리를 재는 방법이라 스케일링이 필수다
- ④ 무리에 의미를 붙이는 건 사람이다

Q9 Level별 코드 작성하기

옆에 주피터 노트북을 열고, 복사 버튼으로 코드를 붙여넣어 따라 해보세요. 쉬운 것부터 실전(캐글 공공데이터)까지 3단계입니다.

코드 해석 ▼을 누르면 한 줄씩 쉽게 풀어 줍니다.

LEVEL 1 단순 — 내장 데이터(iris)로 3무리 나누기

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

X = load_iris().data          # 붓꽃 특성 4개 (정답인 품종은 안 씀)
km = KMeans(n_clusters=3, random_state=0, n_init=10)
labels = km.fit_predict(X)    # 각 꽃이 몇 번 무리인지 (0,1,2)
print(labels[:10])
```

```
from sklearn... import load_iris / KMeans
```

사이킷런에서 '붓꽃 데이터'와 'K-means 도구' 두 개를 꺼내 옵니다. 필요한 도구를 맨 위에서 불러오는 습관이에요.

```
X = load_iris().data
```

붓꽃 150송이의 크기 4개(꽃잎·꽃받침의 길이·너비)만 가져옵니다. 정답인 품종은 **일부러 안 씁니다** — 정답 없이 묶는 게 군집화니까요.

```
km = KMeans(n_clusters=3, random_state=0, n_init=10)
```

"3무리로 나눠줘"라는 도구를 만듭니다. **random_state=0**은 매번 같은 결과가 나오게 고정, **n_init=10**은 대장(중심) 위치를 10번 바꿔 시도해 그중 제일 좋은 걸 고르라는 뜻이에요.

```
labels = km.fit_predict(X)
```

배우면서(fit) 동시에 각 꽃에게 무리 번호(0·1·2)를 붙여(predict) 줍니다. 정답이 없으니 predict 대신 **fit_predict**를 씁니다.

```
print(labels[:10])
```

앞 10송이가 몇 번 무리인지 찍어서 확인합니다. `[:10]`은 '앞에서 10개'라는 뜻.

LEVEL 2 복잡한 데이터 — 특성 13개 와인, 스케일링 + 실루엣 점수

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X = load_wine().data # 와인 성분 13개 (단위가 제각각)
X_s = StandardScaler().fit_transform(X) # 같은 잣대로 맞추기 (거리 기반이라 필수)

km = KMeans(n_clusters=3, random_state=0, n_init=10)
labels = km.fit_predict(X_s)

print('무리별 개수:', np.bincount(labels))
print('실루엣 점수:', round(silhouette_score(X_s, labels), 3))
```

```
import ... (5줄)
```

배열 계산(numpy), 와인 데이터, 스케일러, K-means, 실루엣 점수 — 이번엔 도구가 더 많습니다.

```
X = load_wine().data
```

와인 178명의 성분 **13가지**를 가져옵니다. iris(4개)보다 특성이 훨씬 많아 더 실전에 가까워요.

```
X_s = StandardScaler().fit_transform(X)
```

성분마다 숫자 크기가 판판입니다(알코올 12 vs 마그네슘 100대). 모두 **평균 0·표준편차 1**의 같은 잣대로 바꿉니다. K-means는 거리를 재기 때문에 이걸 빼먹으면 큰 숫자가 결과를 지배해요.

```
labels = km.fit_predict(X_s)
```

원본 X가 아니라 **스케일한 X_s**를 넣어 3무리로 나눕니다. (여기서 X를 넣으면 잘못!)

```
np.bincount(labels)
```

각 무리에 와인이 몇 병씩 들어갔는지 셉니다.

```
silhouette_score(X_s, labels)
```

무리가 얼마나 잘 나뉘었는지 **-1~1** 점수로 알려줍니다. 1에 가까울수록 무리가 뚜렷하게 갈린 것.


```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# 공개 미리 URL로 바로 불러오기 (Kaggle 'Mall Customers')
url = 'https://raw.githubusercontent.com/tirthajyoti/Machine-Learning-with-Python/master/Datasets/Mall_Customers.csv'
df = pd.read_csv(url) # 고객 200명 (다운로드 불필요)
X = df[['Annual Income (k$)', 'Spending Score (1-100)']] # 연소득·소비점수
X_s = StandardScaler().fit_transform(X)

km = KMeans(n_clusters=5, random_state=0, n_init=10)
df['segment'] = km.fit_predict(X_s) # 고객을 5유형으로

print(df.groupby('segment')[['Annual Income (k$)', 'Spending Score (1-100)']].mean().round(1))
```

```
import pandas as pd ...
```

표를 다루는 pandas, 스케일러, K-means를 가져옵니다. 실제 데이터는 **표(CSV)**로 오니 pandas가 필요해요.

```
url = '...Mall_Customers.csv' → pd.read_csv(url)
```

쇼핑몰 고객 200명 데이터를 **공개 URL에서 바로** 읽습니다. 캐글 사이트 자체는 로그인 필요하지만, 이런 고전 데이터는 **GitHub 미리 주소**가 있어 다운로드 없이 불러올 수 있어요.

```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

표의 여러 열 중 '연소득'과 '소비점수' **두 열만** 골라 씁니다. (대괄호 두 개 `[[ ]]` 는 '여러 열 고르기')

```
X_s = StandardScaler().fit_transform(X)
```

소득(수십~수백)과 점수(1~100)는 크기가 다르니 같은 잣대로 맞춥니다.

```
df['segment'] = km.fit_predict(X_s)
```

고객을 5유형으로 나눠 그 번호를 표에 **'segment' 열**로 새로 붙입니다.

```
df.groupby('segment')[...].mean()
```

유형별로 소득·소비점수의 평균을 봅니다. 그러면 "고소득·고소비형", "저소득·고소비형"처럼 **유형에 이름을 붙일 수** 있어요 — 이름 붙이는 건 사람 몫!